ARCHITECTURE PLAN — AUTO-UPDATES · ROLLBACK · BACKUP

MBUMAH HARDWARE POS

A 12-month reliability architecture covering three pillars: GitHub-driven updates that install only while stores are dormant, a one-click rollback button that reverts code and data to the last known-good release, and automatic database backups that survive a crash.

Prepared for: Sam & the Mbumah Team

Prepared by the Development Team — baseline v2.8.0

OCTOBER 2026 · REV 1.0

Table of Contents

1 Executive Summary & Current State 2

2.1 Key features and why they matter . . . 3

2.2 Step-by-step implementation actions . . . 3

2.3 Recommended tools and libraries . . . 4

2.4 Timeline and success metrics . . . 4

3.1 Key features and why they matter . . . 5

3.2 Step-by-step implementation actions . . . 6

3.3 Recommended tools and libraries . . . 6

3.4 Timeline and success metrics . . . 7

4.1 Key features and why they matter . . . 7

4.2 Step-by-step implementation actions . . . 8

4.3 Recommended tools and libraries . . . 8

4.4 Timeline and success metrics . . . 9

5 Cross-Cutting Design: Security, Performance, Usability 10

5.1 Closing the named gaps . . . 10

6 Delivery Roadmap & Governance 11

7 Assumptions, Risks & Immediate Next Steps 12

1 Executive Summary & Current State

This document is the reliability architecture plan for the Mbumah Hardware POS, the point-of-sale and ERP system running across five hardware stores in Kenya on a cloud deployment (Vercel application, Neon PostgreSQL database) plus a one-click laptop deployment kit for counter machines. It answers three requests from the business: first, that the system keep itself current by watching the GitHub repository and installing new releases automatically when the shops are dormant; second, that a rollback button exists so any release can be reverted to the previous stable state in minutes; and third, that the database backs itself up automatically so a crash never means lost sales, lost stock records or lost tax documents. Each pillar is specified with its key features, step-by-step implementation actions, recommended tools, a phased timeline and measurable success metrics.

The plan assumes a small development team of two to three developers with intermediate skills in GitHub-based delivery and database administration, committed to build and test over a twelve-month horizon, and it deliberately builds on infrastructure Mbumah already pays for: GitHub, Vercel, Neon, Resend and Twilio. Today the system has no structured rollback mechanism and no automated backup solution; updates are applied by hand and differ from store to store. Version 2.8.0 has just standardised the application surface, which makes this the right moment to automate the operational layer around it. The three headline targets the whole plan is engineered against are shown below.

≤ 5 min
Rollback time objective (RTO) for any release, app + config

≤ 5 min
Data loss window (RPO) on cloud via WAL archiving and PITR

0
Version drift across the five stores within 7 days of a release

Area	Today (baseline v2.8.0)	Where this plan takes it
Code delivery	Manual git pull and installer re-run per laptop; Vercel deploys main automatically but nothing coordinates the stores with the cloud	Release channel on GitHub Releases; update orchestrator installs during dormancy windows with canary rollout and a kill switch
Rollback	None structured - recovering means reinstalling an older build by hand	One-click Rollback button in Admin, auto-rollback guard on failed health checks, matched database restore point per release
Backups	No laptop backups at all; cloud relies on provider retention with no verified restore path	Scheduled plus pre-migration plus crash-triggered backups, encrypted off-site copies, monthly automated restore tests
Visibility	Health endpoint reports server status only	Health endpoint also exposes version, last backup age and update ledger; admins get WhatsApp/email notices for every operational event

Table 1 - Current state versus target operating model

> **How the three pillars interlock.** Every automatic update first takes a backup, then records a ledger entry binding the new version, the database migration state and the backup identifier together. That ledger entry is exactly what the rollback button consumes, so an update is always one click from a verified restore point, and the backup system is exercised by the release process itself rather than sitting unused until a disaster.

2.1 Key features and why they matter

Manual updates are the largest source of avoidable operational risk in the five-store estate today. When each laptop is updated by hand, stores drift onto different versions, security fixes wait for someone to find the time, and every update competes with trading hours. The goal of this pillar is an update pipeline that monitors the GitHub repository, packages each release with a checksum, installs it only while a store is dormant (default window 22:00 to 04:30 East Africa Time, and only when no shift is open), verifies itself with a smoke test, and reports the outcome to admins on WhatsApp and email. The cloud deployment keeps its existing Vercel Git integration, so merging to main remains the single act of publishing, while tagged GitHub Releases become the one source of truth every store converges on.

Feature	What it does	Why it is important
Release channel	Every release is tagged (semver), built by CI and published to GitHub Releases with changelog notes and a SHA-256 checksum file	Gives all stores and the cloud one signed source of truth; makes updates reproducible and auditable
Repo monitoring	A small updater service polls the GitHub Releases API every 10 minutes, with a webhook receiver as the fast path when available	Polling fallback means a lost webhook never silently stalls updates
Dormancy detector	Checks for open shifts, in-progress sales and running background jobs before applying anything; defers and notifies if the store is active	Guarantees updates never interrupt a sale - the top business rule
Pre-flight checks	Verifies disk space, power state, database integrity and takes a mandatory pre-update backup before touching the running system	A failed pre-flight aborts safely instead of leaving a half-updated till
Staged rollout	Canary store receives the release first; the remaining stores follow only after the canary passes its smoke test for 24 hours	Caps the blast radius of a bad release at one store
Maintenance mode	A banner on the login and POS screens states the update window; the API refuses writes for the two to three minutes of the swap	Staff are never surprised by a restarting system
Update ledger	Every apply is recorded: version, timestamp, migration state, backup identifier, smoke result and who or what triggered it	Feeds the rollback button, troubleshooting and compliance
Kill switch	One environment flag freezes all automatic updates (for example over the December peak or during an incident)	The business stays in control during its busiest trading periods

Table 2 - Feature set for the automatic update pipeline

2.2 Step-by-step implementation actions

Step	Action	Window	Output
1	Release engineering: adopt strict semver tags, generate changelogs, publish GitHub Releases with artifact checksums from CI	Weeks 1-2	Tagged release pipeline live
2	Expose version and readiness: extend the health endpoint with app version, applied migration state and last backup age	Weeks 3-4	Health endpoint v2 consumed by dashboards
3	Build the laptop updater script: check releases, pre-flight, backup, download, verify checksum, extract to releases folder, run migrations, swap the current symlink, restart, smoke test	Weeks 5-8	Update-Mbumah-POS.ps1/.sh v1 (manual trigger)
4	Add dormancy detection and scheduling via Task Scheduler with skip-and-notify behaviour when the store is active	Weeks 9-10	Hands-free dormant-window updates
5	Formalise the cloud path: branch protection, release tags only, Vercel automatic deploy and its built-in instant rollback	Weeks 11-12	Cloud release runbook
6	Canary rollout plus notifications: designate one store as canary, send WhatsApp/email notices before, on success and on failure	Months 4-6	Staged rollout to all five stores
7	Harden: artifact signing, update success dashboard, auto-defer during calendar peak windows, multi-store orchestration	Months 7-12	Signed artifacts and fleet dashboard

Table 3 - Implementation steps for Pillar 1

2.3 Recommended tools and libraries

Tool / library	Role in this pillar
GitHub Actions + Releases API	CI, release packaging, checksums, release metadata consumed by the updater
Vercel Git integration	Cloud deployments on merge; deployment history and instant promotion for rollback
semver + Octokit (Node)	Version comparison and GitHub API calls inside the updater service
PowerShell 5.1 + Task Scheduler	Windows laptop updater and scheduling, matching the existing deployment kit constraints
pm2 / systemd / docker compose	Service supervision and restart after the binary swap on each platform
Resend + Twilio (existing)	Update notifications to admins on WhatsApp, SMS and email
changesets (optional)	Changelog generation from commit history

Table 4 - Tooling choices for Pillar 1

2.4 Timeline and success metrics

Phase	Scope	Exit criteria
0-3 months	Steps 1-5: release channel, health surface, updater v1, dormancy scheduling, cloud runbook	First fully automatic update applied on one store with zero trading-hour impact

Phase	Scope	Exit criteria
3-6 months	Steps 6-7: canary rollout, notification matrix, signing spike	All five stores tracking releases within 7 days; update success rate at or above 99%
6-12 months	Fleet dashboard, peak-season auto-defer, orchestration and drills	Version drift permanently zero; security patches live within 72 hours

Table 5 - Phased timeline for Pillar 1

Metric	Target	How it is measured
Update adoption lag	Under 24 h canary, under 7 days all stores	Ledger timestamps versus release date
Update success rate	At or above 99%	Ledger outcomes per month
Downtime per update	Under 5 minutes per store	Health check gaps during the window
Version drift	0 stores behind after 7 days	Weekly fleet version report
Security patch lag	Under 72 h from advisory to deployed	Advisory date versus ledger entry

Table 6 - Success metrics for Pillar 1

3.1 Key features and why they matter

A bad release is the most common self-inflicted outage in any continuously updated system, and today recovering from one means manually reinstalling an older build on affected machines. The rollback pillar turns that recovery into a controlled, seconds-long operation available to administrators from the web UI, and it is deliberately symmetrical with Pillar 1: because every update records its version, migration state and backup identifier in the ledger, rollback simply restores that matched triple instead of guessing. The design also covers the harder half of rollback that most teams forget - the database - through an expand-and-contract migration policy, so that reverting code never requires a destructive schema reversal.

Feature	What it does	Why it is important
Rollback triples	Every update writes (version, migration state, backup ID) to the ledger before and after applying	Rollback always restores a matched code-plus-data pair, not a guess
Admin Rollback button	Admin, System, Updates: history table with a Rollback action per entry; SUPER_ADMIN or STORE_OWNER only; confirmation dialog captures a reason; writes to the audit log	One controlled click for the business, with accountability built in
Cloud rollback	Vercel instant promotion of the previous deployment via API or dashboard	Seconds to revert bad code with zero schema change involved
Laptop rollback	Capistrano-style releases folder with the last three builds and a current symlink; Rollback-Mbumah-POS script flips the link, restarts and re-runs the smoke test	A till recovers in minutes without an engineer visit
Schema safety policy	Expand-and-contract: additive migrations ship first, destructive changes one release later; failed migrations pinned with prisma migrate resolve	Code can always be rolled back without a destructive schema reversal

Feature	What it does	Why it is important
Data restore pairing	For changes that alter data semantics, rollback pairs with a Neon point-in-time restore to the recorded backup ID (last resort, requires owner approval)	Even data-level incidents have a rehearsed path back
Auto-rollback guard	If three consecutive post-update health or smoke checks fail, the updater automatically reverts to the previous release and alerts	Overnight failures self-heal before staff arrive
Quarterly drills	Scripted rollback exercise in staging every quarter, timed and logged	The button is proven to work before the day it is needed

Table 7 - Feature set for the rollback pillar

3.2 Step-by-step implementation actions

Step	Action	Window	Output
1	Write the rollback policy: when a rollback is allowed, who approves, how data changes are treated, and the drill calendar	Weeks 1-2	Signed-off policy document
2	Laptop mechanism: releases folder layout, keep-last-three builds, Rollback-Mbumah-POS script (symlink flip, restart, smoke test)	Weeks 3-6	Rollback script shipped in the kit
3	Admin UI and API: updates ledger endpoint, Rollback action with role-gated auth, confirmation dialog and audit logging	Weeks 5-8	One-click rollback in Admin
4	Wire cloud rollback: server route calls the Vercel deployment API to promote the previous build for cloud-originated releases	Weeks 9-10	Unified rollback for both platforms
5	Auto-rollback guard in the updater smoke stage; pair PITR restore runbook with the backup pillar	Months 4-6	Self-healing overnight updates
6	Run and time the first two quarterly drills in staging; publish results to the metrics dashboard	Months 4-6	Drill 1 and 2 complete with timings

Table 8 - Implementation steps for Pillar 2

3.3 Recommended tools and libraries

Tool / library	Role in this pillar
Vercel REST API	Programmatic promotion of the previous cloud deployment from the Admin backend
Prisma Migrate	Migration application, resolve-on-failure and migration state recorded in the ledger
Neon point-in-time restore	Database recovery to the exact minute recorded in a rollback triple
Symlinked releases (laptop)	Instant build switching with three generations retained on disk
Zod + session RBAC middleware	Role-gated, validated rollback endpoint matching the existing admin patterns

Tool / library	Role in this pillar
shadcn/ui AlertDialog + audit log	Confirmation UX with captured reason, writing to the immutable audit trail

Table 9 - Tooling choices for Pillar 2

3.4 Timeline and success metrics

Phase	Scope	Exit criteria
0-3 months	Policy, laptop rollback script, Admin button MVP	A rollback completed end to end in staging in under 5 minutes
3-6 months	Auto-rollback guard, PITR pairing, drills 1-2	Overnight bad release self-reverts; drill timings recorded
6-12 months	Chaos testing, SLO dashboard, per-store orchestration	Rollback SLO met in 4 of 4 drills; fewer than 5% of releases ever need rollback

Table 10 - Phased timeline for Pillar 2

Metric	Target	How it is measured
RTO - application rollback	Under 5 minutes	Drill timings and real incidents
RTO - with data restore	Under 30 minutes	PITR rehearsed restore
Rollback success rate	At or above 99%	Ledger outcomes
Releases needing rollback	Under 5% and trending down	Ledger per month

Table 11 - Success metrics for Pillar 2

4.1 Key features and why they matter

The database is the business: sales, customer credit, stock movements and the KRA tax trail all live in it. Today the cloud database benefits from provider retention that nobody has ever tested restoring from, while the laptop deployment kit writes its SQLite database to a single local file with no protection at all - one failed disk or corrupted update could erase a store's transaction history. This pillar installs layered, automated backups with verified restores: continuous WAL archiving on the cloud, real-time replication plus scheduled snapshots on laptops, a mandatory backup gate inside every update, and a crash hook that captures a final safety copy before the system restarts itself.

Feature	What it does	Why it is important
Cloud PITR	Continuous WAL archiving on Neon with the retention window confirmed and documented by the team	Any minute of the last days is recoverable; underpins rollback pairing
Nightly logical dumps	Scheduled pg_dump from CI or Vercel Cron, encrypted, uploaded to object storage (Cloudflare R2 or S3)	Independent copy outside the provider, cheap insurance against account-level mistakes

Feature	What it does	Why it is important
Laptop replication	Litestream streams the SQLite WAL to a second local path and an optional cloud bucket; a nightly consistent snapshot via the better-sqlite3 backup API	Turns the highest-risk unguarded store of data into a protected one
Pre-migration gate	The updater refuses to run migrations unless a fresh backup exists and is recorded in the ledger	Every update starts from a known-good copy of the data
Crash-triggered backup	Global uncaught-exception handler snapshots the database, then a supervisor (pm2, NSSM or Task Scheduler) restarts the service and the health gate verifies recovery	A crash ends with the newest possible data captured, not a corrupt file
One-command restore	Restore-Mbumah-POS script and documented runbook for cloud and laptop; monthly automated restore-to-scratch test	Recovery is rehearsed, not improvised during an outage
Monitoring and alerts	Health endpoint reports last-backup age; WhatsApp/email alert when a backup fails or exceeds 26 hours; uninstaller never deletes backups	Silent backup failure - the classic disaster - becomes impossible to miss

Table 12 - Feature set for the backup pillar

4.2 Step-by-step implementation actions

Step	Action	Window	Output
1	Inventory every data store (cloud Postgres, laptop SQLite, file uploads); confirm and document the Neon retention window; move credentials into a secrets store	Weeks 1-2	Data inventory and signed retention policy
2	Nightly encrypted pg_dump to object storage from scheduled CI; record backup IDs in the update ledger	Weeks 3-4	Off-site cloud copies verified
3	Integrate Litestream plus nightly snapshots into the laptop kit (installer, updater and uninstaller paths)	Weeks 5-8	Laptop data continuously protected
4	Add the pre-migration backup gate to the update pipeline	Weeks 5-8	Updates blocked without a fresh backup
5	Crash hooks and supervised restart; verify by killing the process under load in staging	Weeks 9-10	Crash recovery demonstrated
6	Restore automation: one-command restore script, first restore drill, health endpoint backup-freshness check	Weeks 11-12	First timed restore completed
7	Monthly automated restore tests, alert routing, seven-year archive tier for KRA record-keeping, annual DR game day	Months 4-12	Compliance-grade archive and drills

Table 13 - Implementation steps for Pillar 3

4.3 Recommended tools and libraries

Tool / library	Role in this pillar
Neon PITR (WAL archiving)	Continuous cloud point-in-time recovery, no application code required
pg_dump / pg_restore	Portable logical backups independent of the provider
Litestream	Real-time SQLite WAL replication to local and object storage on every laptop
better-sqlite3 backup API	Consistent nightly snapshots from the running application
Cloudflare R2 or AWS S3 + restic	Encrypted off-site storage with lifecycle rules for the archive tier
GitHub Actions cron / Vercel Cron	Scheduling of dumps, restore tests and freshness checks
pm2 / NSSM / Task Scheduler + Sentry	Supervision, automatic restart and crash capture on laptops and cloud

Table 14 - Tooling choices for Pillar 3

4.4 Timeline and success metrics

Phase	Scope	Exit criteria
0-3 months	Steps 1-6: inventory, cloud dumps, laptop replication, pre-migration gate, crash hooks, first restore drill	A timed restore of a real backup completed; backup age visible in health
3-6 months	Monthly automated restore tests, alert routing, Litestream tuned per store	Backup success at or above 99.5%; failed backup alerts within 15 minutes
6-12 months	Seven-year archive tier, cross-region copy, annual DR game day	Regional-loss scenario rehearsed end to end

Table 15 - Phased timeline for Pillar 3

Metric	Target	How it is measured
RPO - cloud	Under 5 minutes	WAL archive lag
RPO - laptops	Under 1 minute local, under 24 h off-site	Litestream metrics and snapshot log
Backup job success	At or above 99.5%	Scheduled job results
Restore time	Under 30 minutes full store	Quarterly drill timings
Data-loss incidents	Zero	Incident log

Table 16 - Success metrics for Pillar 3

5 Cross-Cutting Design: Security, Performance, Usability

The three pillars share infrastructure, and several qualities must hold across all of them rather than inside any single one. This chapter fixes those cross-cutting decisions once so the implementation stays consistent. Security treats the update and rollback surfaces as privileged operations with their own audit trail. Performance confines all heavy work to dormancy windows and keeps steady-state overhead near zero. Usability applies the vocabulary and patterns already proven in v2.8.0 - honest statuses, confirmation dialogs, visible previews - to the new operational features. Each decision below is a requirement, not a preference, and the roadmap in Chapter 6 assumes all of them.

Concern	Design decision
Role-gated operations	Update apply, rollback, PITR and backup download require SUPER_ADMIN or STORE_OWNER sessions; every call writes to the immutable audit log with actor and reason
Artifact integrity	Releases carry SHA-256 checksums from CI and are verified before install; signing (cosign or similar) arrives in the 6-12 month hardening phase
Secrets hygiene	Gateway and database credentials live in the platform environment or a vault, never in the repository; the v2.8.0 env documentation is the checklist
Dormancy-first performance	Migrations, dumps, replication compaction and downloads run only in the update window; Litestream steady-state overhead is milliseconds
Honest status vocabulary	Operational events use the same Sent/Failed/Deferred conventions the client approved for vouchers, so staff never see a fake success
Confirm before destructive action	Rollback, restore and backup deletion always open a confirmation dialog that captures a reason, matching the v2.8.0 delete pattern

Table 17 - Cross-cutting design decisions

5.1 Closing the named gaps

The brief for this plan called out three systemic gaps explicitly: insufficient error handling, underdeveloped testing protocols and the absence of user notifications. Each is closed structurally rather than case by case, and each closure is observable in the metrics tables of Chapters 2 to 4. The error-handling gap is addressed by global handlers and health gates at every stage boundary, following the dead-letter queue pattern already present in the codebase. The testing gap is addressed by the CI gates that already run on every pull request, extended with a post-update smoke suite and quarterly drills. The notification gap is addressed by the matrix below, which guarantees every operational event reaches the right people on a channel they already use.

Named gap	Consequence today	How this plan closes it
Lack of error handling	Failures surface as blank screens or silent corruption	Global exception handlers, pre-flight checks, health gates after every stage, circuit breaker stops repeated failed updates
Underdeveloped testing protocols	Regressions reach stores before anyone notices	CI gates per pull request, canary store, post-update smoke suite (login, sale, inventory load, receipt print), quarterly rollback and restore drills

Named gap	Consequence today	How this plan closes it
Absence of user notifications	Staff and owners learn about updates and outages by accident	Notification matrix: 24-hour notice, at-window start, success, failure, rollback and backup alerts on WhatsApp, email and the in-app banner

Table 18 - Structural closure of the three named gaps

Event	Channels	Audience	Timing
Update scheduled	Banner + email	All staff, admins	24 hours before window
Update applying / done / failed	WhatsApp + email + banner	Admins, store managers	At the window
Rollback executed (manual or auto)	WhatsApp + email	Admins, owner	Within 5 minutes of the event
Backup failed or stale	WhatsApp + email	Admins	Within 15 minutes of detection
Monthly restore test result	Email	Admins	First business day of the month

Table 19 - Operational notification matrix

6 Delivery Roadmap & Governance

The roadmap sequences the three pillars so that early months remove the worst risk - unprotected data - while the machinery for safe, frequent releases is assembled in parallel. Months 0 to 3 deliver the release channel, the first automatic update on a single store, the laptop backup layer and the rollback MVP. Months 3 to 6 scale the update pipeline to all five stores, add self-healing and notification routing, and run the first drills. Months 6 to 12 are hardening: signing, dashboards, the KRA-aligned archive tier and a full disaster-recovery game day. Governance is intentionally light: a fortnightly release train, one versioning ritual, and quarterly drills with published timings.

Phase	Pillar 1 - Updates	Pillar 2 - Rollback	Pillar 3 - Backups
0-3 months	Release channel, health surface, updater v1, dormancy scheduling	Rollback policy, laptop rollback script, Admin button MVP	Data inventory, cloud dumps, laptop replication, crash hooks, first restore drill
3-6 months	Canary rollout, notifications, signing spike	Auto-rollback guard, PITR pairing, drills 1-2	Monthly restore tests, alert routing, archive tuning
6-12 months	Fleet dashboard, peak auto-defer, orchestration	Chaos testing, SLO dashboard	Seven-year archive, cross-region copy, annual DR game day

Table 20 - Master roadmap across the three pillars

Versioning governance continues the existing chain: package.json, the central version module and the four consumer surfaces (footer, login page, health endpoint and OpenAPI) bump together on every release, so the fleet dashboard and the rollback ledger always agree on what is running where. The definition of done for

every roadmap item is that it is deployed by the pipeline it automates, verified by a drill or smoke test, and observable through the health endpoint or dashboard - no pillar is complete on code review alone. Sign-off for each phase rests with the business owner, using the same acceptance-checklist format introduced for v2.8.0.

> **First 90 days in one sentence.** By the end of week 12 every laptop backs itself up continuously, one store receives updates automatically in its dormancy window, an administrator can roll the system back from the Admin screen, and a real restore has been timed end to end.

7 Assumptions, Risks & Immediate Next Steps

The plan assumes the application under discussion is the Mbumah Hardware POS, a development team of two to three people with intermediate skills in GitHub-based integration and database management, and a committed build-and-test horizon of twelve months, matching the phase structure above. It further assumes the current stack remains in place - GitHub, Vercel, Neon PostgreSQL on the cloud, the Node and SQLite laptop kit, Resend and Twilio for messaging - and that store laptops have power protection proportional to their importance, since no backup survives a dead disk on the only copy. Where an assumption breaks, the risk register below names the mitigation in advance.

Risk	Likelihood	Impact	Mitigation
Schema rollback against drifted data	Medium	High	Expand-and-contract policy; PITR pairing; destructive changes one release behind code
Laptop offline or powered off in the update window	Medium	Medium	Catch-up check on next boot; staleness alert after 48 hours
Gateway rate limits or outages mute notifications	Low	Medium	Dual-channel alerts (WhatsApp plus email); in-app banner as the always-present channel
SQLite replication fidelity concerns	Low	Medium	Litestream is purpose-built for SQLite; monthly restore tests prove the copies
Gateway credentials never configured (vouchers precedent)	Medium	Medium	Health endpoint config check; env checklist in the client guide; alerts on unconfigured state

Table 21 - Risk register with mitigations

Six actions start the plan, in order. First, verify and document the Neon retention window and take the first scripted nightly dump in week one, because the cloud backup is the fastest win available. Second, stand up the release engineering pipeline - tags, changelogs, checksums - in weeks one and two, since every other automation consumes it. Third, ship the updates ledger endpoint and the Admin rollback button MVP in the following sprint, using the existing admin patterns from v2.8.0. Fourth, run a Litestream spike on one store laptop and measure replication lag under real trading load. Fifth, execute restore drill number one and publish the timing to set the baseline for the RTO metric. Sixth, schedule the first quarterly rollback drill and put the drill calendar in front of the business owner for sign-off.

> **Closing note.** This plan was prepared against baseline v2.8.0 in October 2026. It is written to be executed by the small team it describes, one fortnightly release train at a time, and to leave the business in control - updates happen only when the shops say so, rollback is one click with accountability, and no crash can take the books with it.